■ JavaScript LeetCode Cheatsheet

Number Constants · ASCII · Map · Set · Array · String · Tricks & Templates

■ Number Constants

Constant	Value	Use Case
`Number.MAX_SAFE_INTEGER`	9,007,199,254,740,991	Max safe integer in JS
`Number.MIN_SAFE_INTEGER`	-9,007,199,254,740,991	Min safe integer in JS
`Infinity`	+∞	Init min-tracking variable
`-Infinity`	-∞	Init max-tracking variable
`Number.EPSILON`	~2.22e-16	Floating point precision check
`Number.MAX_VALUE`	~1.8e+308	Largest possible float

```
let minVal = Infinity; // track minimum — start high
let maxVal = -Infinity; // track maximum — start low
Number.MAX_SAFE_INTEGER // 9007199254740991
Number.isSafeInteger(x) // true if |x| <= 2^53-1
```

■ ASCII Reference

Character	Decimal Range
`A – Z (Uppercase)`	65 – 90
`a – z (Lowercase)`	97 – 122
`0 – 9 (Digits)`	48 – 57
`Space`	32
`Newline \n`	10

Quick Maths	Result
`'A'.charCodeAt(0)`	65
`'a'.charCodeAt(0)`	97
`'0'.charCodeAt(0)`	48
`'z' - 'a'`	25
`97 - 65`	32 (case gap)

charCodeAt & fromCharCode

```
'A'.charCodeAt(0) // → 65
String.fromCharCode(65) // → 'A'

// 0-25 index for a-z (most common LC pattern!)
const idx = ch.charCodeAt(0) - 97; // 'a'→0 'z'→25
String.fromCharCode(idx + 97); // 0→'a' 25→'z'
String.fromCharCode(idx + 65); // 0→'A' 25→'Z'
```

The 32 Trick — Toggle Case

```
'a'.charCodeAt(0) - 32 // → 65 = 'A' (lower → upper)
'A'.charCodeAt(0) + 32 // → 97 = 'a' (upper → lower)

// XOR 32 = elegant toggle (works both directions!)
String.fromCharCode('a'.charCodeAt(0) ^ 32) // → 'A'
String.fromCharCode('A'.charCodeAt(0) ^ 32) // → 'a'
```

■■ Map Methods

Method / Property	Returns	Notes
`map.set(key, val)`	→ Map	Add/update; chainable

Method / Property	Returns	Notes
`map.get(key)`	→ value	undefined if key missing
`map.has(key)`	→ boolean	O(1) lookup
`map.delete(key)`	→ boolean	true if key existed
`map.clear()`	void	Remove ALL entries
`map.size`	→ number	Property — NOT .length!
`map.keys()`	Iterator	All keys
`map.values()`	Iterator	All values
`map.entries()`	Iterator	[key, value] pairs

```
// Frequency count — most used Map pattern
const freq = new Map();
for (const c of s) freq.set(c, (freq.get(c) ?? 0) + 1);

// Iteration (note: value comes FIRST in forEach!)
for (const [k, v] of map) { }
map.forEach((value, key) => { });

// Default value shorthand
map.get(key) ?? 0
```

■■ Set Methods

Method / Property	Returns	Notes
`set.add(val)`	→ Set	Chainable; silently ignores duplicates
`set.has(val)`	→ boolean	O(1) — use instead of arr.includes!
`set.delete(val)`	→ boolean	true if value existed
`set.clear()`	void	Remove ALL
`set.size`	→ number	Property — NOT .length!

```
const unique = [...new Set(arr)]; // deduplicate array

// Set operations
const union = new Set([...a, ...b]);
const inter = new Set([...a].filter(x => b.has(x)));
const diff = new Set([...a].filter(x => !b.has(x)));
```

✂■ Array Methods

Method	Mutates?	Returns	Notes
`push(v)`	■ YES	new length	Add to end
`pop()`	■ YES	removed val	Remove from end
`shift()`	■ YES	removed val	Remove from start — O(n)!
`unshift(v)`	■ YES	new length	Add to start — O(n)!
`splice(i, n, ...v)`	■ YES	removed[]	Remove / insert anywhere
`sort((a,b)=>a-b)`	■ YES	→ Array	MUST pass comparator for numbers!
`reverse()`	■ YES	→ Array	
`fill(v, s, e)`	■ YES	→ Array	Fill slice [s, e)
`slice(s, e)`	■ NO	→ new Array	Subarray [s, e) — negative ok
`concat(arr2)`	■ NO	→ new Array	

Method	Mutates?	Returns	Notes
`map(fn)`	■ NO	→ new Array	Transform each element
`filter(fn)`	■ NO	→ new Array	Keep matching elements
`reduce(fn, init)`	■ NO	→ value	Accumulate to single value
`flat(depth)`	■ NO	→ new Array	Flatten nested arrays
`includes(v)`	■ NO	→ boolean	O(n) linear search
`indexOf(v)`	■ NO	→ index | -1	First occurrence
`find(fn)`	■ NO	→ value	First matching value
`findIndex(fn)`	■ NO	→ index	First matching index
`some(fn)`	■ NO	→ boolean	true if ANY element matches
`every(fn)`	■ NO	→ boolean	true if ALL elements match

Create Arrays

```
Array(n).fill(0) // [0, 0, ..., 0]
Array.from({length: n}, (_, i) => i) // [0, 1, ..., n-1]
[...Array(n).keys()] // [0, 1, ..., n-1]

// Safe 2D array — each row is INDEPENDENT
Array.from({length: m}, () => Array(n).fill(0))

// ■ WRONG — all rows share the same reference!
Array(m).fill(Array(n).fill(0))
```

■ String Methods

Method	Notes
`str.length`	Property, not a method
`str[i] / str.charAt(i)`	Access single character
`str.charCodeAt(i)`	Char → ASCII number
`String.fromCharCode(n)`	ASCII number → Char
`str.indexOf(sub)`	First index of sub, or -1
`str.lastIndexOf(sub)`	Last index of sub, or -1
`str.includes(sub)`	→ true / false
`str.startsWith(pre)`	→ true / false
`str.endsWith(suf)`	→ true / false
`str.slice(s, e)`	[s, e) — negatives ok — strings are IMMUTABLE
`str.split(sep)`	→ Array. split('') gives char array
`str.replace(s, r)`	Replaces FIRST match only
`str.replaceAll(s, r)`	Replaces ALL matches
`str.toUpperCase()`	
`str.toLowerCase()`	
`str.trim()`	Remove whitespace from both ends
`str.padStart(n, ch)`	Pad left until length is n
`str.padEnd(n, ch)`	Pad right until length is n
`str.repeat(n)`	Repeat string n times

```
str.split('').reverse().join('') // Reverse a string
```

■ Clever LC Tricks & Templates

1 Clamp Index — Never Go Out of Bounds

```
Math.max(0, Math.min(n - 1, k)) // clamp k to [0, n-1]
const i = Math.min(n - 1, k); // cap at max index
const i = Math.max(0, k); // floor at 0
```

2 Two Pointers Template

```
let l = 0, r = arr.length - 1;
while (l < r) {
if (condition) l++;
else r--;
}
```

3 Sliding Window Template

```
let l = 0, windowSum = 0;
for (let r = 0; r < arr.length; r++) {
windowSum += arr[r];
while (windowSum > target) windowSum -= arr[l++];
// valid window is [l, r]
}
```

4 Binary Search — Safe Mid (No Overflow)

```
let lo = 0, hi = arr.length - 1;
while (lo <= hi) {
const mid = lo + ((hi - lo) >> 1); // floor, no overflow
if (arr[mid] === target) return mid;
else if (arr[mid] < target) lo = mid + 1;
else hi = mid - 1;
}
return -1;
```

5 XOR Tricks

```
a ^ a === 0 // same number cancels itself
a ^ 0 === a // XOR identity
// Find single number (all others appear twice):
arr.reduce((acc, n) => acc ^ n, 0)

[a, b] = [b, a]; // swap with destructuring (cleanest)
a ^= b; b ^= a; a ^= b; // XOR swap (no temp variable)
```

6 Bit Tricks

```
x >> 1 // floor(x / 2)
x << 1 // x * 2
x & 1 // 0 = even, 1 = odd
x & (x - 1) // clear lowest set bit
(x & (x-1)) === 0 // true if x is a power of 2
~~x or x | 0 // Math.trunc fast (positives only)
```

7 Modular Arithmetic

```
const MOD = 1e9 + 7;
result = (result % MOD + MOD) % MOD; // always positive

// Handles negative n in mod:
((n % m) + m) % m
```

8

Sort Gotchas

```
// Default sort is LEXICOGRAPHIC — always wrong for numbers!
[10, 9, 2].sort() // [10, 2, 9] WRONG!
[10, 9, 2].sort((a, b) => a-b) // [2, 9, 10] correct

arr.sort((a, b) => a.val - b.val) // by property
arr.sort((a, b) => a.localeCompare(b)) // strings
```

9

GCD, LCM & Math Utils

```
const gcd = (a, b) => b === 0 ? a : gcd(b, a % b);
const lcm = (a, b) => (a / gcd(a, b)) * b;

Math.max(...arr) // max of array (caution: large arrays)
Math.min(...arr) // min of array
arr.reduce((a,b) => Math.max(a,b), -Infinity) // safe max
```

10

Nullish Coalescing & Optional Chaining

```
val ?? default // use default only if val is null/undefined
obj?.prop // safe property access
arr?.[i] // safe array index
fn?.() // safe function call
```

■■ Quick Reference Card

Need	Code	Result
Char → ASCII	'A'.charCodeAt(0)	65
ASCII → Char	String.fromCharCode(65)	'A'
a→0 z→25 index	ch.charCodeAt(0) - 97	0 … 25
Rebuild from index	String.fromCharCode(idx + 97)	'a'…'z'
Toggle case (XOR)	String.fromCharCode(c.charCodeAt(0)^32)	
Unique array	[...new Set(arr)]	
Freq count	m.set(k, (m.get(k) ?? 0) + 1)	
Clamp index	Math.max(0, Math.min(n-1, k))	
Int divide	~~(a / b) or Math.floor(a/b)	
Is odd?	n & 1	1=odd 0=even
Power of 2?	n & (n-1) === 0	true / false
Array max/min	Math.max(...arr)	
Swap values	[a, b] = [b, a]	
Reverse string	s.split('').reverse().join('')	
Safe 2D array	Array.from({length:m},()=>Array(n).fill(0))	
GCD	const gcd=(a,b)=>b?gcd(b,a%b):a	
Always positive mod	((n % m) + m) % m	
Single number XOR	arr.reduce((a,b) => a^b, 0)	
Palindrome check	s === s.split('').reverse().join('')	